СОГЛАСОВАНО	УТВЕРЖДАЮ
Доцент КафОИ ФИТ	Студент 4 курс
_____ М. В. Рутман	_____ Е. А. Тестова
24.10.2025	24.10.2025

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

РАСШИРЕНИЕ CONFLICT-FREE REPLICATED DATA TYPE НАД JSONB ДЛЯ POSTGRESQL

Техническое задание

Лист утверждения

RU.777001.00001-01 93 01-ЛУ

Электронный документ

Листов 22

Подпись и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № подл

Представители предприятия-разработчика

Главный инженер

_____ Е. А. Тестова

24.10.2025

Начальник отдела

_____ Е. А. Тестова

24.10.2025

Руководитель разработки

_____ Е. А. Тестова

24.10.2025

Исполнитель

_____ Е. А. Тестова

24.10.2025

Нормоконтролер

_____С. В. Желободько

24.10.2025

СОГЛАСОВАНО

Доцент КафОИ ФИТ

_____ М. В. Рутман

24.10.2025

Ассистент КафОИ ФИТ

_____ А. В. Винтер

24.10.2025

Т

2025

УТВЕРЖДЕНО

RU.777001.00001-01 93 01-ЛУ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

РАСШИРЕНИЕ CONFLICT-FREE REPLICATED DATA TYPE НАД JSONB ДЛЯ POSTGRESQL

Техническое задание

RU.777001.00001-01 93 01

Электронный документ

Листов 22

Выполнила:

Тестова Екатерина 22204

Подпись и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата	Инв. № подл

Т

2025

СОДЕРЖАНИЕ

1 Введение ... 5
1.1 Наименование ... 5
1.2 Краткая характеристика области применения ... 5
1.3 Краткая характеристика объекта ... 5
2 Основание для разработки ... 6
2.1 Основание для разработки ... 6
2.2 Наименование и условное обозначение разработки ... 6
3 Назначение разработки ... 7
3.1 Функциональное назначение ... 7
3.2 Эксплуатационное назначение ... 7
3.3 Целевая аудитория ... 7
4 Требования к программе ... 8
4.1 Требования к функциональным характеристикам ... 8
4.1.1 Требования к составу выполняемых функций ... 8
4.1.2 Требования к организации входных и выходных данных ... 9
4.1.3 Требования к временным характеристикам ... 10
4.2 Требования к надежности ... 10
4.2.1 Обеспечение устойчивого функционирования ... 10
4.2.2 Контроль входной и выходной информации ... 11
4.2.3 Время восстановления после отказа ... 11
4.3 Условия эксплуатации ... 11
4.3.1 Условия эксплуатации ... 11
4.3.2 Вид обслуживания ... 12
4.3.3 Необходимое количество и квалификация персонала ... 12
4.4 Требования к составу и параметрам технических средств ... 12
4.5 Требования к информационной и программной совместимости ... 13
4.6 Требования к маркировке и упаковке ... 13

4.7 Требования к транспортированию и хранению ... 13

4.7.1 Условия транспортирования ... 13

4.7.2 Условия и места хранения ... 14

4.7.3 Условия складирования и сроки ... 14

5 Требования к программной документации ... 15

6 Технико-экономические показатели ... 16

6.1 Ориентировочная экономическая эффективность ... 16

6.2 Предполагаемая годовая потребность ... 16

6.3 Экономические преимущества разработки ... 16

7 Стадии и этапы разработки ... 18

8 Порядок контроля и приемки ... 22

1 ВВЕДЕНИЕ

1.1 Наименование

Короткое наименование: «CRDT над JSONB».

Полное наименование: Расширение Conflict-free Replicated Data Type над JSONB для PostgreSQL.

1.2 Краткая характеристика области применения

Областью применения разработки является обеспечение бесконфликтной репликации данных в распределенных и облачных системах баз данных, использующих PostgreSQL, при работе с полуструктурированными данными.

Основное назначение разработки - предоставление механизма гарантированной сходимости реплик в многомастерных конфигурациях PostgreSQL за счет реализации Conflict-free Replicated Data Type (CRDT) для типа данных JSONB, что особенно актуально для финансового сектора, банковского процессинга и систем управления конфигурациями.

1.3 Краткая характеристика объекта

Расширение PostgreSQL представляет собой новый тип данных, внешне совместимый с JSONB, но реализующий внутреннюю логику CRDT для автоматического бесконфликтного слияния изменений в распределенных системах. Оно позволяет выполнять операцию вставки в любой массив внутри объекта JSONB, при этом обеспечивая гарантированную согласованность реплик в многомастерной репликации без необходимости ручного разрешения конфликтов. Особенностью типа является поддержка упорядочивания элементов в подмассивах по времени их вставки, что гарантирует eventual consistency - конечную сходимость данных до идентичного состояния на всех узлах системы. Это позволяет применять решение в критически важных системах, таких как банковский процессинг и управление настройками пользователей.

2 ОСНОВАНИЕ ДЛЯ РАЗРАБОТКИ

2.1 Основание для разработки

Основанием для проведения разработки является Распоряжение № 0753 от «24» октября 2025, согласованное с Рутманым Михаилом Валерьевичем, доцент КафОИ ФИТ (далее Заказчик), утвержденный Тестовой Екатериной Александровной, студент (далее Разработчик 1).

2.2 Наименование и условное обозначение разработки

Наименование темы разработки – «Разработка расширения CRDT над JSONB для PostgreSQL».

Условное обозначение темы разработки (шифр темы) – «CRDT-2025».

3 НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1 Функциональное назначение

Разрабатываемое расширение PostgreSQL «JSONB-CRDT» предназначено для обеспечения бесконфликтной репликации полуструктурированных данных в распределенных и облачных системах баз данных.

Расширение обеспечивает:

1) Предоставление типа данных, внешне совместимого с JSONB, но с внутренней CRDT-логикой.
2) Автоматическое слияние конфликтующих изменений при многомастерной репликации.
3) Поддержку операции вставки элемента в массив внутри объекта JSONB.
4) Гарантированную сходимость реплик до идентичного состояния (eventual consistency).
5) Сохранение семантики порядка элементов в массивах на основе времени вставки.

Функциональное назначение расширения заключается в устранении необходимости ручного разрешения конфликтов данных на уровне приложения и обеспечении прозрачной для разработчика синхронизации в распределенных системах, что повышает надежность и отказоустойчивость при работе с полуструктурированными данными.

3.2 Эксплуатационное назначение

Программное средство предназначено для использования в распределенных и облачных системах на базе PostgreSQL, где требуется обеспечение бесконфликтной репликации полуструктурированных данных.

3.3 Целевая аудитория

Целевой аудиторией являются:

1) Разработчики распределенных систем баз данных, использующих PostgreSQL.
2) Администраторы СУБД, обеспечивающие работу многомастерных кластеров.
3) Команды разработки финансовых систем и банковских приложений.
4) Специалисты по облачным технологиям и микросервисным архитектурам.

4 ТРЕБОВАНИЯ К ПРОГРАММЕ

4.1 Требования к функциональным характеристикам

4.1.1 Требования к составу выполняемых функций

Все операции изменения данных допускаются только через специализированные функции расширения. Изменение структуры объекта напрямую (например, через UPDATE ... SET column = ...) запрещено — все модификации должны осуществляться через API расширения для обеспечения корректности сходимости.

Функциональная возможность: «Создание поля типа crdt_jsonb».

Необходимые действия:

1) Выполнить SQL-запрос CREATE EXTENTION crdt_jsonb.

2) Выполнить SQL-запрос CREATE TABLE <table_name> (<column_name> crdt_jsonb).

3) Настроить между всеми узлами двунаправленную логическую репликацию (например, create publication pub1_name for table table_name; create subscription sub1_name connection 'host=node2_host port=node2_port dbname=postgres' publication pub2_name with (create_slot=true, copy_data = false, origin = none, binary = true))

4) Создать объект командой insert into table_name values(1, '') или insert into table_name values(0, '{"key" : "value"}').

Функциональная возможность: «Добавление элемента в массив по указанному пути».

Необходимые действия:

1) Вызвать функцию crdt_jsonb_append(<column>, <path>, <value>) (например, update table_name set data = crdt_jsonb_append(data, '{"key" : "value"}', '{}') where id = 0), где:

- <column> — имя столбца типа crdt_jsonb.
- <path> — строка пути внутри JSONB-объекта до массива, в который нужно вставить новое значение.
- <value> — JSON-значение, которое необходимо добавить.

2) Убедиться, что <path> указывает на существующий массив внутри объекта.

3) Убедиться, что операция возвращает обновлённое значение типа crdt_jsonb.

Функциональная возможность: «Чтение значения crdt_jsonb».

Необходимые действия:

1) Выполнить SQL-запрос SELECT <column> FROM <table_name> WHERE <condition>;

2) Убедиться, что возвращаемое значение является валидным JSON-объектом, соответствующим типу jsonb.

Функциональная возможность: «Конвертация crdt_jsonb в стандартный jsonb» Необходимые действия:

1) Вызвать функцию get_jsonb(<column>).

2) Убедиться, что возвращаемое значение является корректным значением типа jsonb, в котором все массивы содержат только значения без меток времени и внутренних полей CRDT.

Функциональная возможность: «Синхронизация состояний между репликами».

Необходимые действия:

1) На одном узле выполнить одну или несколько операций crdt_jsonb_append с разными путями и значениями.

2) На другом узле выполнить SELECT <column> FROM <table_name>;

3) Убедиться, что состояние crdt_jsonb на целевом узле идентично состоянию на исходном узле (включая порядок элементов в массивах), даже при одновременных изменениях на разных узлах.

4.1.2 Требования к организации входных и выходных данных

Единственными входными данными для операций расширения являются:

- Значение типа crdt_jsonb (существующее состояние объекта).
- Строка, задающая путь внутри JSONB-объекта.
- JSON-значение (любой тип: строка, число, объект, массив, null), которое необходимо добавить в массив по указанному пути.

Входные данные должны быть переданы исключительно через вызовы функций расширения: crdt_jsonb_append(<column>, <path>, <value>) — основная операция вставки.

Ограничения на входные данные:

- Значение < path > может указывать только на массив, иначе вернется ошибка.
- Значение <value> должно быть корректным JSON-объектом (валидный JSON).
- Операции, не использующие функции расширения (например, прямое присваивание column = '{}'::jsonb), не обрабатываются расширением и не обеспечивают свойств CRDT — такие действия не считаются входными данными для системы синхронизации.

Выходными данными являются:

- Обновлённое значение типа crdt_jsonb.
- Значение типа jsonb, возвращаемое функцией get_jsonb().

Выходные данные не содержат внутренних метаданных CRDT.

4.1.3 Требования к временным характеристикам

Функциональная возможность: «Создание поля типа crdt_jsonb».

Время выполнения: не более 100 мс, независимо от размера объекта и количества вложенных массивов.

Функциональная возможность: «Добавление элемента в массив по указанному пути» (через crdt_jsonb_append).

Время выполнения: не более 100 мс на локальном узле при нагрузке до 1000 операций в секунду.

Функциональная возможность: «Чтение значения crdt_jsonb».

Время выполнения: не более 10 мс, независимо от размера объекта и количества вложенных массивов.

Функциональная возможность: «Конвертация crdt_jsonb в стандартный jsonb» (через get_jsonb).

Время выполнения: не более 50 мс.

Функциональная возможность: «Синхронизация состояний между репликами».

Время сходимости: все реплики достигают согласованного состояния в течение 500 мс после завершения репликации изменений (при нормальной работе сети и отсутствии задержек в WAL).

4.2 Требования к надежности

4.2.1 Обеспечение устойчивого функционирования

Расширение crdt_jsonb должно обеспечивать устойчивую работу в составе многомастерного кластера PostgreSQL при стандартной нагрузке, включающей интенсивные параллельные операции добавления данных на разных узлах.

Для обеспечения устойчивости функционирования применяются следующие меры:

1) Реализация всех операций через API, исключая прямое изменение состояния CRDT и предотвращая некорректную модификацию данных.

2) Использование метаданных для гарантии сходимости реплик без блокировок и конфликтов при одновременных вставках.

3) Поддержка совместимости с механизмами репликации PostgreSQL (logical replication), обеспечивающими доставку изменений между узлами с сохранением порядка фиксации транзакций.

4) Логирование ошибок выполнения функций расширения через стандартный механизм PostgreSQL.

4.2.2 Контроль входной и выходной информации

Контроль корректности входной информации осуществляется на уровне СУБД. Все данные, передаваемые в функции расширения, проходят строгую валидацию: проверка типа аргументов (crdt_jsonb, text для пути, jsonb для значения);

Выходная информация автоматически преобразуется в стандартный формат jsonb при выборке, исключая внутренние метаданные CRDT. При этом сохраняется целостность структуры и порядок элементов в массивах по времени вставки.

Все ошибки выполнения (некорректный путь, невалидный JSON, попытка модификации вне API) фиксируются в системном логе PostgreSQL и возвращаются как SQL-ошибки с понятным описанием для разработчика или администратора.

4.2.3 Время восстановления после отказа

Расширение не требует дополнительных этапов восстановления. В случае сбоя узла PostgreSQL, на котором работает расширение crdt_jsonb, восстановление выполняется за счет механизмов PostgreSQL.

4.3 Условия эксплуатации

4.3.1 Условия эксплуатации

Расширение crdt_jsonb предназначено для эксплуатации в среде PostgreSQL 16.

Работа расширения требует:

- Доступа к СУБД через стандартный SQL-интерфейс (например, psql, pgAdmin, приложение на JDBC/ODBC).
- Включённой двунаправленной логической репликации (logical replication) для работы в многомастерной конфигурации.
- Стабильного сетевого соединения между узлами кластера для обеспечения своевременной синхронизации изменений.

Расширение не требует специализированного оборудования, внешних сервисов или веб-интерфейса — оно интегрируется непосредственно в PostgreSQL и используется через SQL-запросы.

Поддержка клиентских приложений обеспечивается через любые средства, поддерживающие стандартные драйверы PostgreSQL.

4.3.2 Вид обслуживания

Обслуживание расширения включает плановое обновление версии расширения, мониторинг производительности операций с типом crdt_jsonb, управление логами и регулярное резервное копирование базы данных.

Обновления расширения выполняются администратором путём установки новой версии на каждый узел кластера PostgreSQL с последующей перезагрузкой экземпляра или переподключением сессий.

4.3.3 Необходимое количество и квалификация персонала

Для сопровождения и эксплуатации системы требуется:

1) backend-разработчик (1 человек) — требуемая квалификация: уверенное знание SQL, Си, внутреннего устройства PostgreSQL, опыт написания расширений или патчей для PostgreSQL;

2) тестировщик (QA-инженер, 1 человек) — требуемая квалификация: знания методологий тестирования, умение разрабатывать и выполнять тест-кейсы, использовать инструменты для тестирования (testgres, tap tests, pgbench);

Пользователи системы нуждаются в специальной технической подготовке. Система предназначена для разработчиков, умеющих взаимодействовать с PostgreSQL и его расширениями, а также использовать их.

4.4 Требования к составу и параметрам технических средств

Для использования расширения crdt_jsonb требуется серверное оборудование, соответствующее следующим параметрам:

1) Процессор и память: минимум 2 ядра CPU и 4 ГБ ОЗУ — для обеспечения устойчивой работы при нагрузке до 1000 операций crdt_jsonb_append в секунду на узел.

2) Хранилище: SSD-накопитель с достаточным объёмом для хранения базы данных и журналов транзакций (WAL).

3) Сетевое подключение: стабильное сетевое соединение (минимум 100 бит/с) между узлами кластера для обеспечения своевременной репликации.

4) Программное обеспечение: установленный и настроенный PostgreSQL 16 с включённой двунаправленной логической репликацией и правами на установку расширений.

Расширение не требует клиентских устройств — оно работает исключительно на стороне сервера СУБД. Доступ к данным осуществляется через SQL-клиенты (например, psql, pgAdmin, JDBC/ODBC-приложения), поддерживающие стандартные протоколы PostgreSQL.

4.5 Требования к информационной и программной совместимости

Расширение crdt_jsonb должно функционировать в составе PostgreSQL версии 16. Оно совместимо с механизмом логической репликации и работает через стандартный SQL-интерфейс.

Данные хранятся в формате JSONB, обмен между узлами осуществляется в бинарном виде через WAL. Внутренние метаданные используют временные метки (timestamp).

Расширение реализовано на языке C с использованием официального API PostgreSQL. Не требует сторонних интерпретаторов, фреймворков или клиентских сред.

4.6 Требования к маркировке и упаковке

Дистрибутив расширения публикуется в открытом доступе на GitHub в официальном репозитории.

В состав входят:

1) Исходный код на C с файлами .sql и .control.
2) README.md с инструкцией по установке.

Развертывание выполняется пользователем через docker compose up или ручную установку в PostgreSQL 16.

4.7 Требования к транспортированию и хранению

4.7.1 Условия транспортирования

Все данные, передаваемые в рамках проекта, должны быть защищены от несанкционированного доступа или изменений.

При необходимости сетевой передачи допускается использование только защищённых каналов связи (HTTPS/TLS, SFTP, VPN). При физической передаче через USB-накопители или внешние HDD необходимо зашифровать носители алгоритмом AES-256 и защитить паролем.

Ответственность за сохранность данных при транспортировке лежит на стороне, осуществляющей передачу.

4.7.2 Условия и места хранения

Исходный код должен храниться на GitHub в приватном репозитории, доступ к которому имеют только инженеры-программисты продукта. Основная ветка main должна иметь защиту: запрет прямых изменений, требование pull request, к которому обязателен хотя бы один отзыв.

Все изменения кода должны быть версионированы и отслеживаться в системе контроля версий Git.

4.7.3 Условия складирования и сроки

Исходный код остается на стороне разработки на протяжении всего жизненного цикла продукта.

5 ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

- «Расширение Conflict-free Replicated Data Type над JSONB для PostgreSQL». Техническое задание (ГОСТ 19.201-78).
- «Расширение Conflict-free Replicated Data Type над JSONB для PostgreSQL». Спецификация (ГОСТ 19.202-78).
- «Расширение Conflict-free Replicated Data Type над JSONB для PostgreSQL». Текст программы (ГОСТ 19.401-78).
- «Расширение Conflict-free Replicated Data Type над JSONB для PostgreSQL». Описание программы (ГОСТ 19.402-78).
- «Расширение Conflict-free Replicated Data Type над JSONB для PostgreSQL». Программа и методика испытаний (ГОСТ 19.301-79).
- «Расширение Conflict-free Replicated Data Type над JSONB для PostgreSQL». Руководство оператора (ГОСТ 19.505-79).
- «Расширение Conflict-free Replicated Data Type над JSONB для PostgreSQL». Пояснительная записка (ГОСТ 19.404-79).

6 ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1 Ориентировочная экономическая эффективность

Разработка расширения crdt_jsonb направлена на повышение отказоустойчивости и упрощение разработки приложений с многомастерной репликацией в PostgreSQL.

Внедрение расширения позволяет сократить трудозатраты разработчиков на реализацию логики разрешения конфликтов данных в распределённых системах. Вместо написания кастомной бизнес-логики для обработки дубликатов, потерь изменений или блокировок, разработчики получают готовый тип данных с гарантией сходимости.

Среднее время, затрачиваемое командой на проектирование и тестирование механизма синхронизации, сокращается на 130–140 часов на проект. Уменьшается вероятность ошибок в управлении данными, что снижает расходы на отладку и восстановление в продакшене.

Расширение особенно эффективно для сервисов, работающих в географически распределённых кластерах, где высокая доступность и автономность узлов являются критичными.

6.2 Предполагаемая годовая потребность

Целевая аудитория расширения crdt_jsonb — разработчики и администраторы PostgreSQL, создающие распределённые и облачные приложения с многомастерной репликацией.

Ожидается, что в первый год после публикации расширение будет использовано в 10–15 проектах (внутренние системы, стартапы, исследовательские платформы).

В последующие годы прогнозируется рост числа внедрений на 20–30% в год за счёт повышения осведомлённости о CRDT и расширении практик отказоустойчивых систем.

6.3 Экономические преимущества разработки

Анализ существующих подходов к решению конфликтов в многомастерных PostgreSQL-кластерах показывает следующие ключевые преимущества расширения crdt_jsonb:

1) Отсутствие необходимости в кастомной логике разрешения конфликтов — в отличие от ручной реализации на уровне приложения (триггеры, обработчики, сложные алгоритмы), расширение предоставляет готовый тип данных с гарантированной сходимостью — снижает затраты на разработку и тестирование на 30–50%.

2) Полная интеграция с PostgreSQL — в отличие от внешних решений, расширение работает внутри СУБД без дополнительных компонентов, снижая сложность инфраструктуры, лицензионные и эксплуатационные расходы.

3) Фокус на надёжность на уровне данных — расширение предотвращает конфликты на архитектурном уровне, а не после их возникновения, что уменьшает риски потери данных, простоя и ошибок в продакшене, особенно в критичных средах (научные, образовательные, облачные проекты).

7 СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Стадии и этапы разработки

Таблица 1

Стадии разработки	**Этапы разработки**	**Содержание работ**	**Исполнитель**	**Срок разработки**
1. Техническое задание	Обоснование необходимости разработки программы	Постановка задачи. Сбор исходных материалов. Выбор и обоснование критериев эффективности и качества разрабатываемой программы. Обоснование необходимости проведения научно-исследовательских работ.	Заказчик, Разработчик 1	2 недели
	Научно-исследовательские работы	Определение структуры входных и выходных данных. Предварительный выбор методов решения задач. Обоснование целесообразности применения ранее разработанных программ. Определение требований к техническим средствам. Обоснование принципиальной возможности решения поставленной задачи.	Разработчик 1	2 недели

Таблица 1

Стадии разработки	**Этапы разработки**	**Содержание работ**	**Исполнитель**	**Срок разработки**
	Разработка и утверждение технического задания	Определение требований к программе. Разработка технико-экономического обоснования разработки программы. Определение стадий, этапов и сроков разработки программы и документации на неё. Выбор языков программирования. Определение необходимости проведения научно-исследовательских работ на последующих стадиях. Согласование и утверждение технического задания.	Заказчик, Разработчик 1	1 неделя
2. Эскизный проект	Разработка эскизного проекта	Предварительная разработка структуры входных и выходных данных. Уточнение методов решения задачи. Разработка общего описания алгоритма решения задачи. Разработка технико-экономического обоснования.	Заказчик, Разработчик 1	2 недели
	Утверждение эскизного проекта	Разработка пояснительной записки. Согласование и утверждение эскизного проекта.	Заказчик, Разработчик 1	2 недели

Таблица 1

Стадии разработки	**Этапы разработки**	**Содержание работ**	**Исполнитель**	**Срок разработки**
3. Технический проект	Разработка технического проекта	Уточнение структуры входных и выходных данных. Разработка алгоритма решения задачи. Определение формы представления входных и выходных данных. Определение семантики и синтаксиса языка. Разработка структуры программы. Окончательное определение конфигурации технических средств.	Разработчик 1	3 месяца
	Утверждение технического проекта	Разработка плана мероприятий по разработке и внедрению программ. Разработка пояснительной записки. Согласование и утверждение технического проекта.	Заказчик, Разработчик 1	2 недели
4. Рабочий проект	Разработка программы	Программирование и отладка программы.	Разработчик 1	3 месяца
	Разработка программной документации	Разработка программных документов в соответствии с требованиями ГОСТ 19.101-77.	Заказчик, Разработчик 1	1 месяц

Таблица 1

Стадии разработки	**Этапы разработки**	**Содержание работ**	**Исполнитель**	**Срок разработки**
	Испытания программы	Разработка, согласование и утверждение порядка и методики испытаний. Проведение предварительных государственных, межведомственных, приёмо-сдаточных и других видов испытаний. Корректировка программы и программной документации по результатам испытаний.	Заказчик, Разработчик 1	3 недели
5. Внедрение	Подготовка и передача программы	Подготовка и передача программы и программной документации для сопровождения и (или) изготовления. Оформление и утверждение акта о передаче программы на сопровождение и (или) изготовление. Передача программы в фонд алгоритмов и программ.	Заказчик, Разработчик 1	2 недели

8 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ

Контроль и приемка разработки осуществляются в соответствии с документом «Программа и методика испытаний». По завершению приемки Разработчиком 1 и Заказчиком будет подписан «Акт приемо-сдаточных работ».

Производится проверка корректного выполнения программой заложенных в нее функций, то есть осуществляется функциональное тестирование программы.